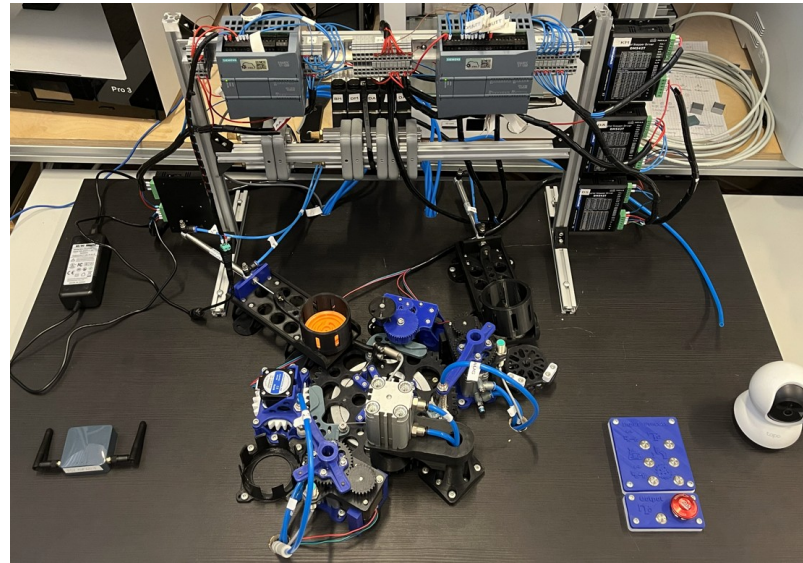


**HOCHSCHULE  
HANNOVER**  
UNIVERSITY OF  
APPLIED SCIENCES  
AND ARTS

–  
*Fakultät II*  
*Maschinenbau und*  
*Bioverfahrenstechnik*



# **Anleitung zur Erstellung eines Digitalen Zwillings**

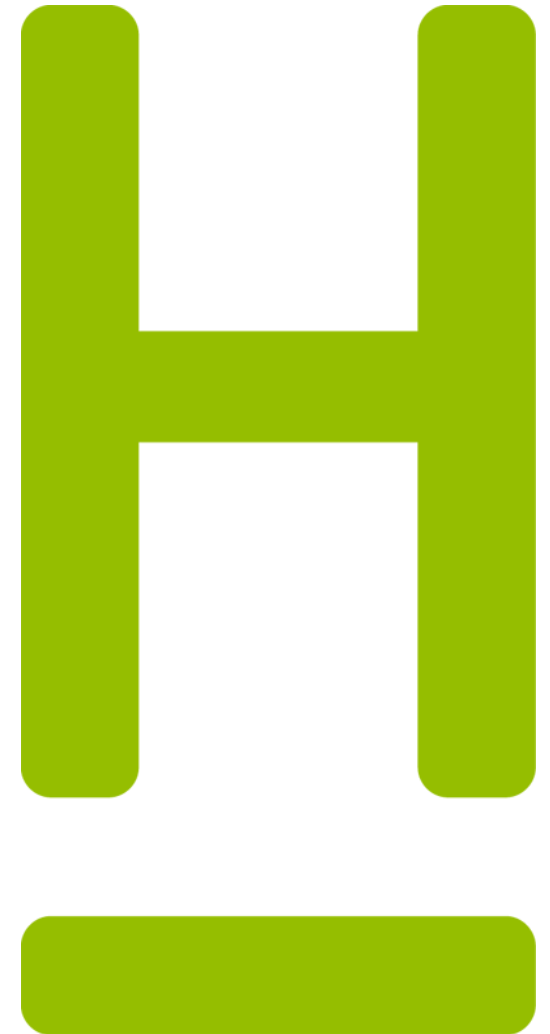
*Projektarbeit von Tobias Küster 1690767*

Betreut von:

**Prof. Diersen**

**Dr. Yübo Wang**

**Herrn Ernst**



# Inhaltsverzeichnis

|                  |                                       |                 |
|------------------|---------------------------------------|-----------------|
| <b>Kapitel 1</b> | Einleitung                            |                 |
| <b>Kapitel 2</b> | Anwendungsfälle und Strukturdiagramme | <i>Folie 4</i>  |
| <b>Kapitel 3</b> | Installation von Isaac Sim            | <i>Folie 8</i>  |
| <b>Kapitel 4</b> | Erste Schritte in Isaac Sim           | <i>Folie 11</i> |
|                  | Oberfläche von Isaac Sim              |                 |
|                  | Bewegliche-/ starre Teile             |                 |
| <b>Kapitel 5</b> | Gelenke und Antriebe                  | <i>Folie 17</i> |
|                  | Dreh- und Schubgelenke mit Antrieb    |                 |
|                  | Verschachtelte Gelenke                |                 |
| <b>Kapitel 6</b> | Zu der Extension                      | <i>Folie 22</i> |
|                  | Ausführung von Quellcode in Isaac Sim |                 |
|                  | Einbinden der Extension               |                 |
| <b>Kapitel 8</b> | Zu TIA Portal                         | <i>Folie 38</i> |



# Einleitung

Die Erweiterung der Anlage „Maze Runner“ ersetzt die mechanische Ausgabestation durch einen Dobot Magician. Die Implementierung ist als lokaler Digitaler Zwilling für den Offline-Betrieb im Hochschullabor konzipiert. Der Datentransfer erfolgt ohne externe Netzwerkabhängigkeiten direkt zwischen der Hardware, dem WSL 2 und NVIDIA Isaac Sim. Die Hardware-Anbindung wird über eine USB-Bridge mittels usbipd realisiert, welche die serielle Schnittstelle des Roboters vom Windows-Host in das Linux-Subsystem (Ubuntu 22.04) tunnelt. Ein Python-Skript fungiert dort als Treiber, extrahiert die Gelenkwinkel J1–J4 und publiziert diese als `sensor_msgs/JointState` über den lokalen ROS 2-DDS-Layer. Die Aktuierung innerhalb der Simulation erfolgt deterministisch über den Action Graph. Ein ROS 2 Subscribe JointState-Node empfängt die Daten und leitet diese an einen Articulation Controller weiter. Durch die Verknüpfung mit der Articulation Root und die Nutzung von `usePathNames` erfolgt die präzise Zuordnung der Winkelwerte auf die Gelenkhierarchie des URDF-Modells. Zur Gewährleistung der physikalischen Stabilität wird die Roboterbasis im URDF über einen `world-link` starr verankert. Um Schreibzugriffe des Action Graphs auf die Gelenk-Transformationen zu ermöglichen und Fehlermeldungen im USD-Layer zu vermeiden, muss die Eigenschaft `Instanceable` (Instanzierung) für alle beweglichen Gelenke im Stage-Tree deaktiviert werden. Dies stellt die funktionale Synchronisation zwischen physischer Hardware und Simulation sicher.



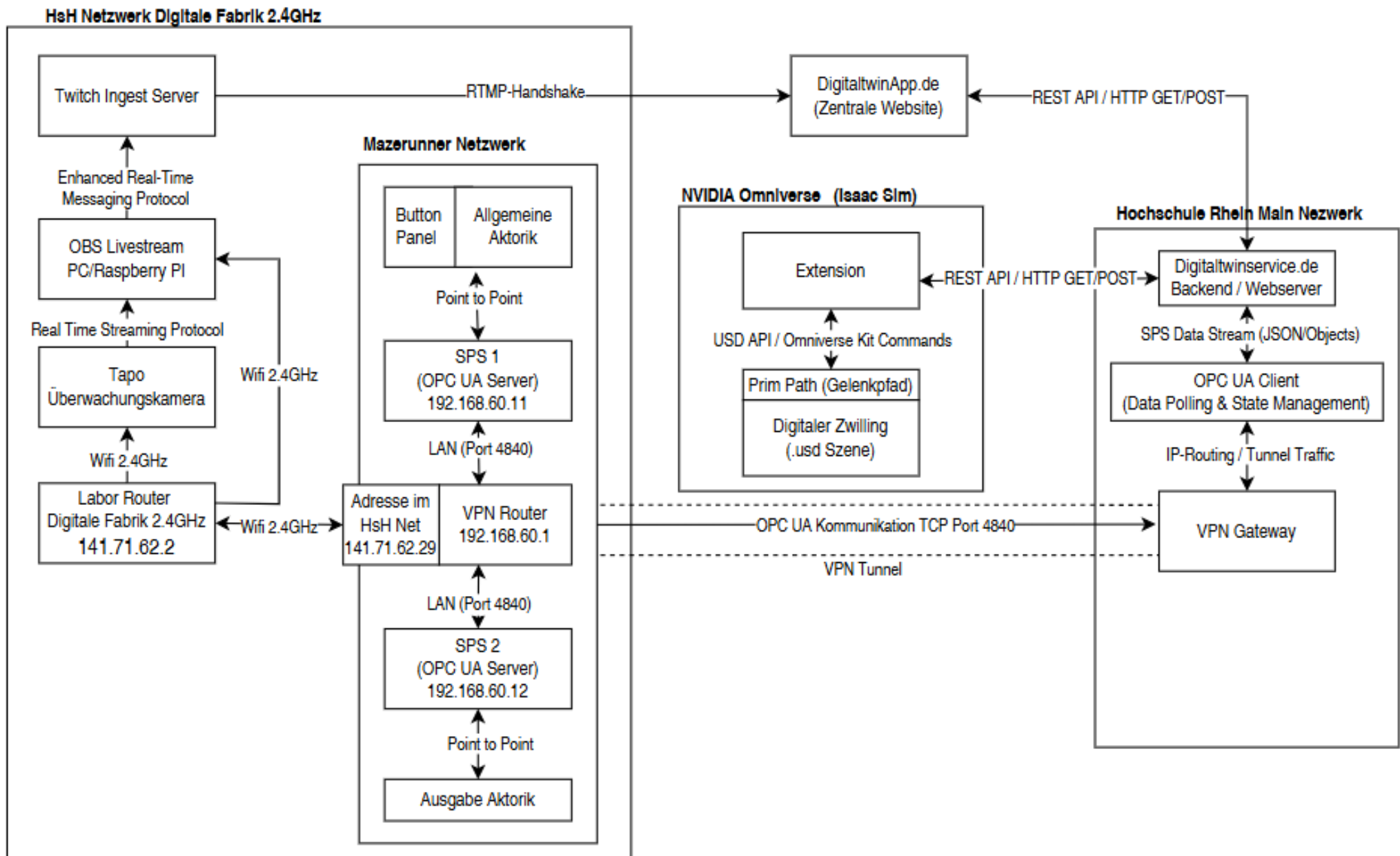
# Use Case Matrix Variable setzen

| Kriterium        | Physikalisch (Labor)               | Lokal an Anlage (OPC UA + Omni Extens.)   | Omni Extens. via Digital Twin Service             |
|------------------|------------------------------------|-------------------------------------------|---------------------------------------------------|
| User-Aktivität   | Taster an der Anlage betätigen     | Schaltfläche im Omniverse HMI betätigen   | Schaltfläche im Omniverse HMI betätigen           |
| Was passiert?    | SPS-Variable direkt intern gesetzt | OPC UA Write Service                      | REST-API Call (HTTP POST) DigitalTwinService      |
| Wo passiert es?  | Innerhalb der SPS                  | Omniverse + lokales Netz                  | Omniverse + Internet/VPN                          |
| Wie passiert es? | Elektrisches Signal                | Omniverse + lokaler OPC UA Server der SPS | HTTPS zu digitaltwinservice.de dann OPCUA via VPN |
| Aktorbewegung    | Reale Hardware                     | Reale Hardware + Omniverse Szene          | Reale Hardware + Omniverse Szene                  |

# Use Case Matrix Variable setzen

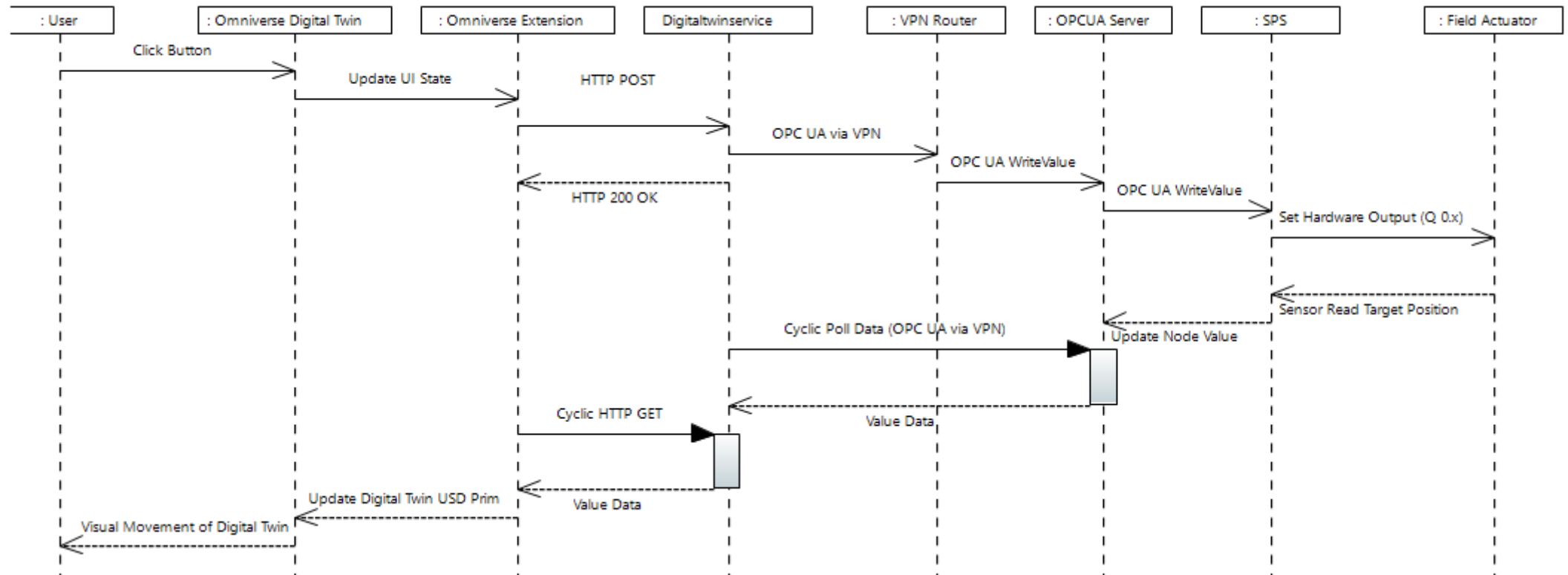
| Kriterium     | Physikalisch (Labor)                                 | Lokal an Anlage (OPC UA + Omni Extens.)          | Omni Extens. via Digital Twin Service     |
|---------------|------------------------------------------------------|--------------------------------------------------|-------------------------------------------|
| Rückmeldung   | Status lokal an der Aktostellung der Anlage sichtbar | Omniverse Szene + ggf. Anlage                    | Omniverse Szene + ggf. Kamera Lifestream  |
| Kommunikation | Direkt SPS-Signal                                    | OPC UA der SPS                                   | HTTP mit X-API-KEY                        |
| Latenz        | < 10 ms                                              | 100-500 ms                                       | 500ms Loop + API                          |
| Fehleranalyse | Hardware vor Ort + TIA Portal                        | API-Statuscodes + Omniverse Konsole + TIA Portal | "TIMEOUT/HTTP" Labels + Omniverse Konsole |
| Fernsteuerung | Nur lokal                                            | Nur im Netz der Anlage                           | Weltweit                                  |

# Internal Block Diagram



# Sequenzdiagramm Variable Setzen

Digital Twin Sequence Diagramm



# WSL Herunterladen & ROS installieren

1. WSL 2 Installieren → Powershell → `wsl --install -d Ubuntu-22.04`
2. Username & password erstellen
3. In ubuntu dann nacheinander folgende Befehle aufrufen:

```
# System-Update
sudo apt update && sudo apt upgrade -y

# Locale auf UTF-8 setzen
sudo apt install locales
sudo locale-gen en_US en_US.UTF-8
sudo update-locale LC_ALL=en_US.UTF-8 LANG=en_US.UTF-8
export LANG=en_US.UTF-8

# ROS 2 Repository hinzufügen
sudo apt install software-properties-common
sudo add-apt-repository universe
sudo apt update && sudo apt install curl -y
sudo curl -sSL https://raw.githubusercontent.com/ros/rosdistro/master/ros.key -o /usr/share/keyrings/ros-archive-keyring.gpg
echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/ros-archive-keyring.gpg] http://packages.ros.org/ros2
Oder
echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/ros-archive-keyring.gpg] http://packages.ros.org/ros2
# ROS 2 installieren (Desktop-Version für alle Tools)
sudo apt update
sudo apt install ros-humble-desktop -y
```



## Dobot an ROS koppeln

```
echo "source /opt/ros/humble/setup.bash" >> ~/.bashrc
source ~/.bashrc
sudo apt update
sudo apt install python3-pip -y
pip3 install pydobot rclpy
nano ~/dobot_ros_bridge.py
sudo usermod -aG dialout „user“
Hier das skript auf der nächsten Folie einfügen →
```

In das schwarze Fenster einfügen, dann Strg + O → Enter dann Strg + X

```
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import JointState
from pydobot import Dobot
import math
import time

class DobotBridge(Node):
    def __init__(self):
        super().__init__('dobot_bridge')
        self.publisher_ = self.create_publisher(JointState, 'joint_states', 10)

        # Hinweis: Im Labor musst du prüfen, ob es /dev/ttyUSB0 oder /dev/ttyS3 ist
        try:
            self.device = Dobot(port='/dev/ttyUSB0')
            self.get_logger().info("Dobot verbunden!")
        except:
            self.get_logger().error("Kein Dobot gefunden - Simulation läuft im Leerlauf")
            self.device = None

        self.timer = self.create_timer(0.05, self.timer_callback) # 20Hz

    def timer_callback(self):
        msg = JointState()
        msg.header.stamp = self.get_clock().now().to_msg()

        # EXAKTE Namen aus deinem URDF
        msg.name = ['joint_1', 'joint_2', 'joint_5', 'joint_6']

        if self.device:
            pose = self.device.pose()
            # Umrechnung Grad -> Radiant
            msg.position = [math.radians(p) for p in [pose[4], pose[5], pose[6], pose[7]]]
        else:
            # Test-Modus ohne Roboter
            msg.position = [0.0, 0.0, 0.0, 0.0]

        self.publisher_.publish(msg)

def main():
    rclpy.init()
    node = DobotBridge()
    rclpy.spin(node)
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```



# USB Driver herunterladen

<https://github.com/dorssel/usbipd-win/releases>



# Dobot auf WSL mappen

Sobald der Dobot per USB angeschlossen ist:

1. PowerShell (Admin) öffnen.
2. `usbipd list` eingeben. Suche die BUSID des Dobots (z.B. 2-3).
3. `usbipd bind --busid <BUSID>` (Nur einmalig nötig).
4. `usbipd attach --busid <BUSID> --distribution Ubuntu-22.04`

Hinweis: Wenn Windows meckert, dass der "WSL-Kernel aktualisiert" werden muss, folge dem Link in der Fehlermeldung.



# Dobot auf WSL mappen

Sobald der Dobot per USB angeschlossen ist:

1. Dobot einschalten
2. Powershell öffnen: `usbipd list` dann `usbipd bind --busid 2-3` (ggf. mit anderen Werten → 2-3 1a86:7523 USB2.0-Serial im Beispiel)
3. USB Port an WSL Durchreichen → Powershell: `usbipd attach --wsl --busid 2-3`
4. WSL-Terminal öffnen. Prüfen, ob der Dobot erkannt wird: `ls /dev/ttyUSB*` (Es sollte `/dev/ttyUSB0` erscheinen).
5. Falls du keine Berechtigung für den Port hast: `sudo chmod 666 /dev/ttyUSB0`.
6. User für serielle Schnittstelle dauerhaft Admin rechte geben: `sudo usermod -a -G dialout $USER`
7. WSL neu starten
8. Skript starten: `python3 ~/dobot_ros_bridge.py`.
9. Du solltest jetzt im Terminal die Meldung "Dobot verbunden!" sehen.



# Startroutine

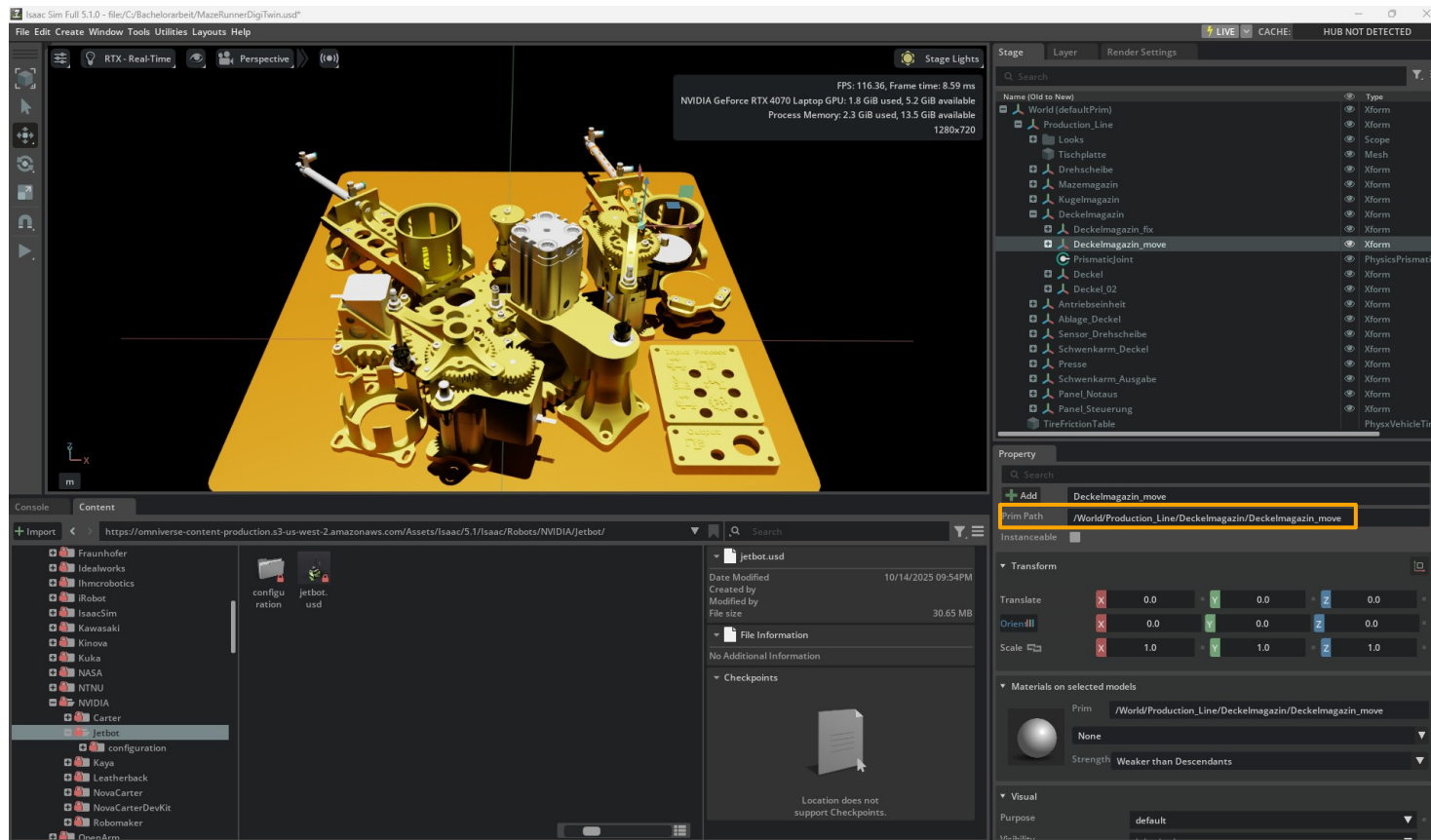
1. Hardware: Dobot an USB -> Windows PowerShell (Admin) -> usbipd attach.
2. WSL: python3 ~/dobot\_ros\_bridge.py (Meldung "Dobot verbunden!" abwarten).
3. **Isaac Sim:** \* URDF-Importer öffnen.
  1. ~/dobot\_magician.urdf wählen.
  2. Haken bei **"Merge Fixed Joints"** und **"Fix Base Link"** setzen.
  3. Haken bei **"Instanceable"** entfernen (falls nötig).
  4. Simulation starten.



# Erste Schritte in Isaac Sim

Stage: Modellansicht  
(Rechtsklick → drehen)

Werkzeuge:  
Verschieben  
Skalieren  
etc.  
Play um die  
Szene zu starten



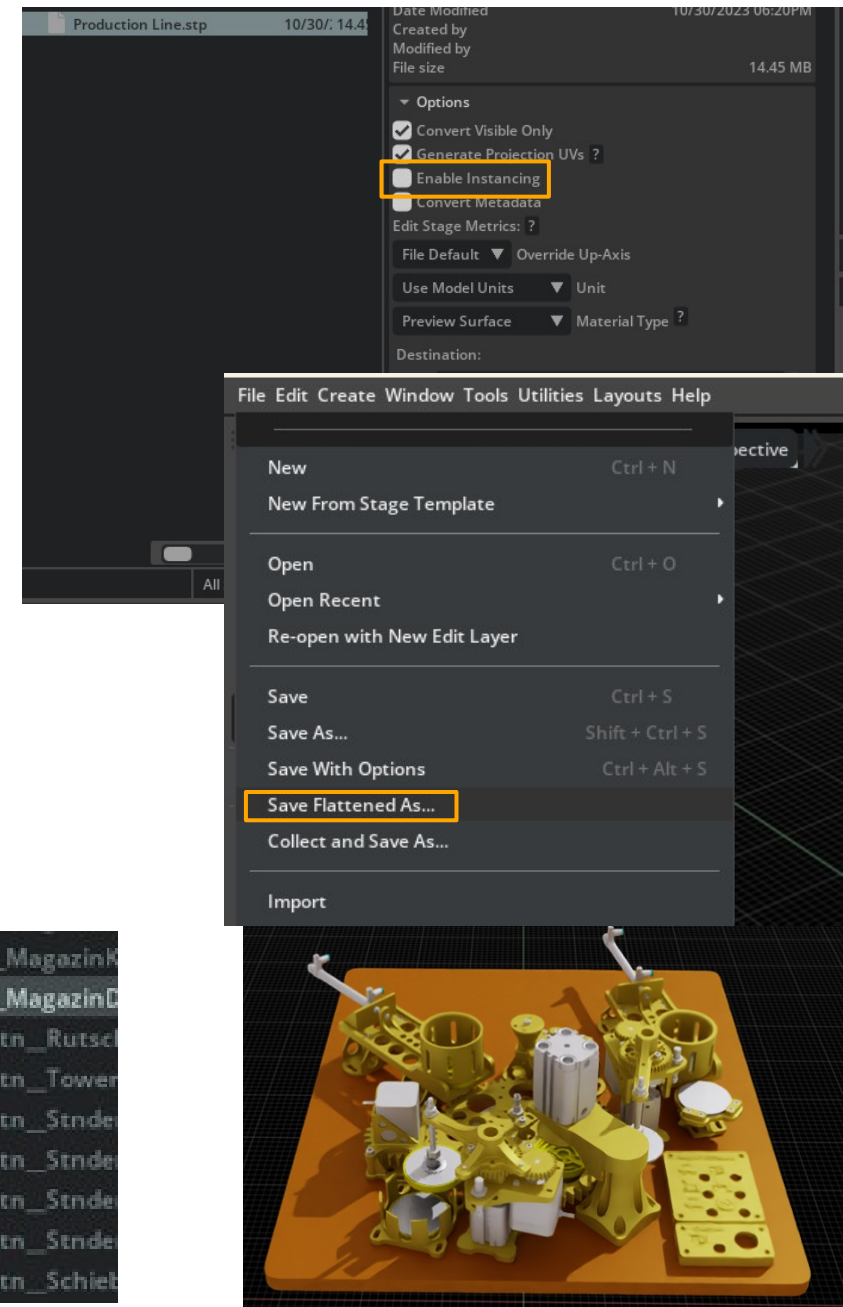
Stage:  
Ordnerstruktur der  
Baugruppen-  
komponenten  
Sowie Beleuchtung  
Material etc.

Property:  
Eigenschaften des  
ausgewählten  
Modells  
Prim Path: ist der  
Pfad zu dem  
ausgewählten  
Objekt

Content: Dateiübersicht von Omniverse  
Dateien oder Lokalen Projekten

# STP-Datei in Isaac Sim öffnen

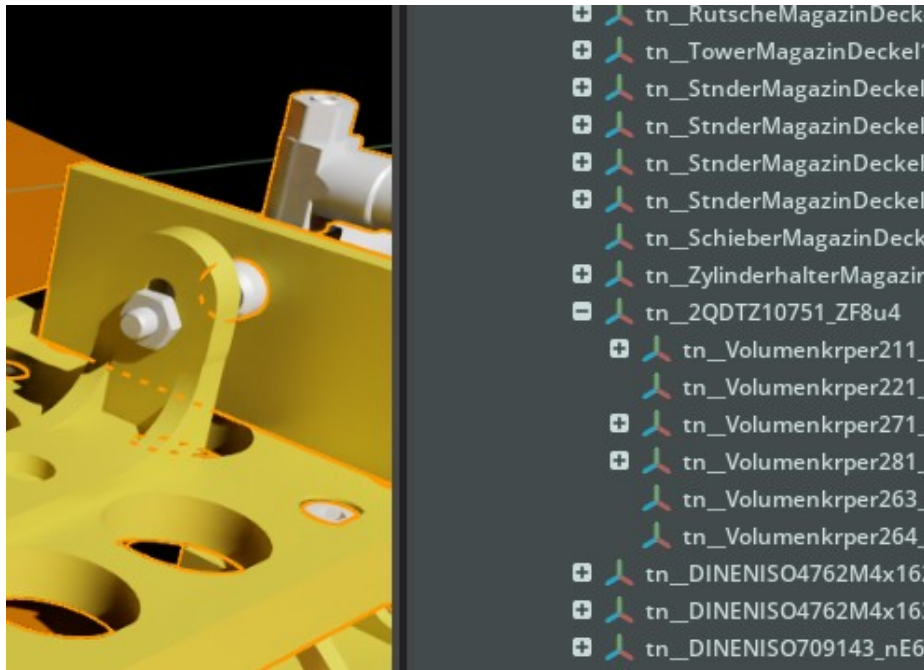
1. File → Import → .STP-Datei auswählen  
Dabei „Enable Instancing“ deaktivieren
2. Um Abhängigkeiten zu anderen Modellen zu löschen:  
File → „Save Flattened As...“  
Sollte dieses ausgegraut sein, vorerst „Save as...“
3. Nach öffnen des gespeicherten Modells sollten die Farben übernommen worden sein
4. **Wichtig:** Im Stage Bereich dürfen weder blaue- noch orangefarbene Pfeile an den Einträgen sein.  
**Grund:** Das würde bedeuten, dass dieses Teil mit einer Datei außerhalb des Projektes synchronisiert ist und nicht veränderbar ist.  
**Prüfen:** Der Name sollte änderbar sein





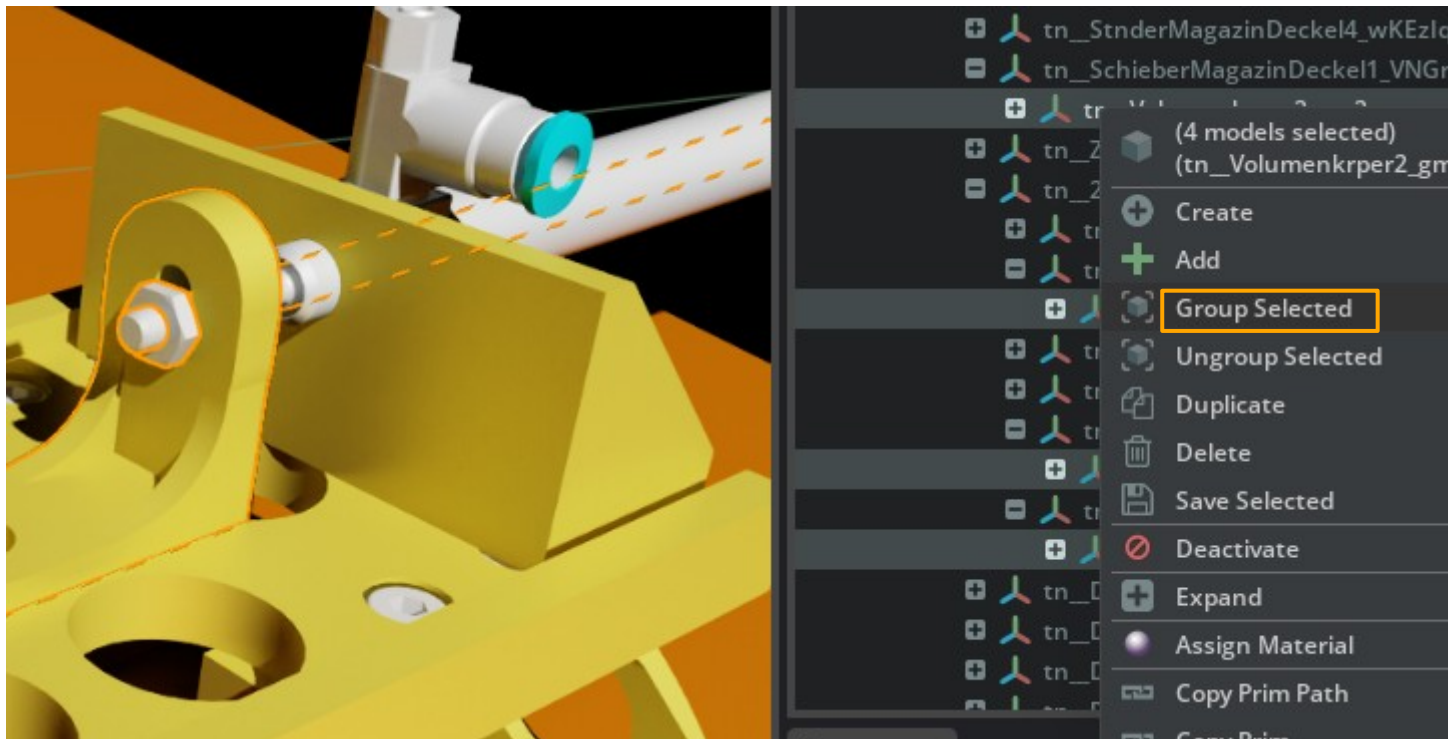
# Starre Gelenkteile am Bsp. Magazin Glasplatte

1. Alle Teile auswählen die starr sein sollen
2. Rechtsklick auf eines der Teile und „Group Selected“ auswählen
3. Die erstellte Gruppe „Magazin\_Glasplatte\_fix“ benennen
4. Die finale Ordnerstruktur sollte wie folgt aussehen:



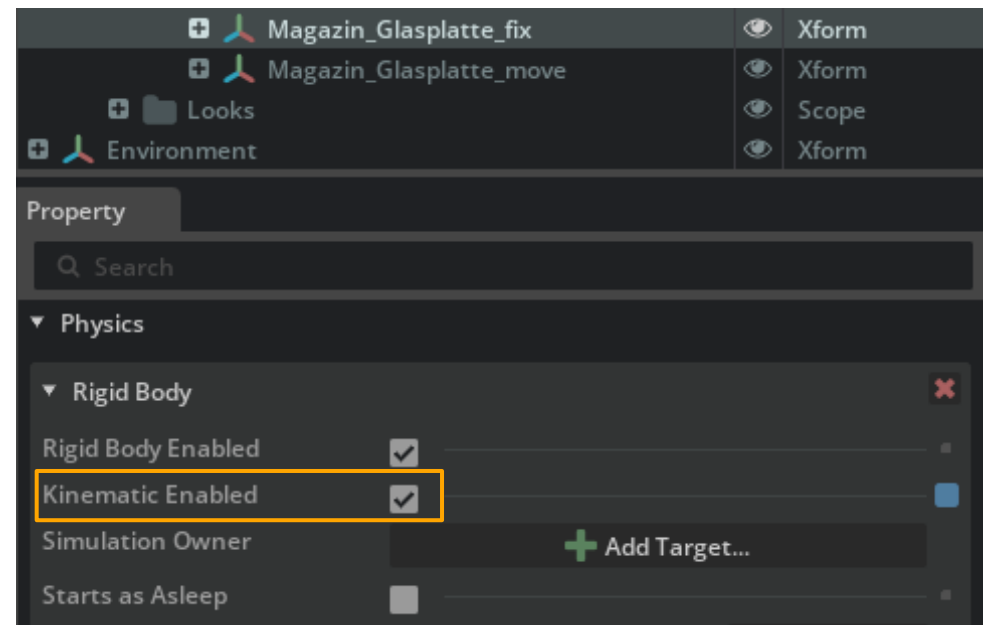
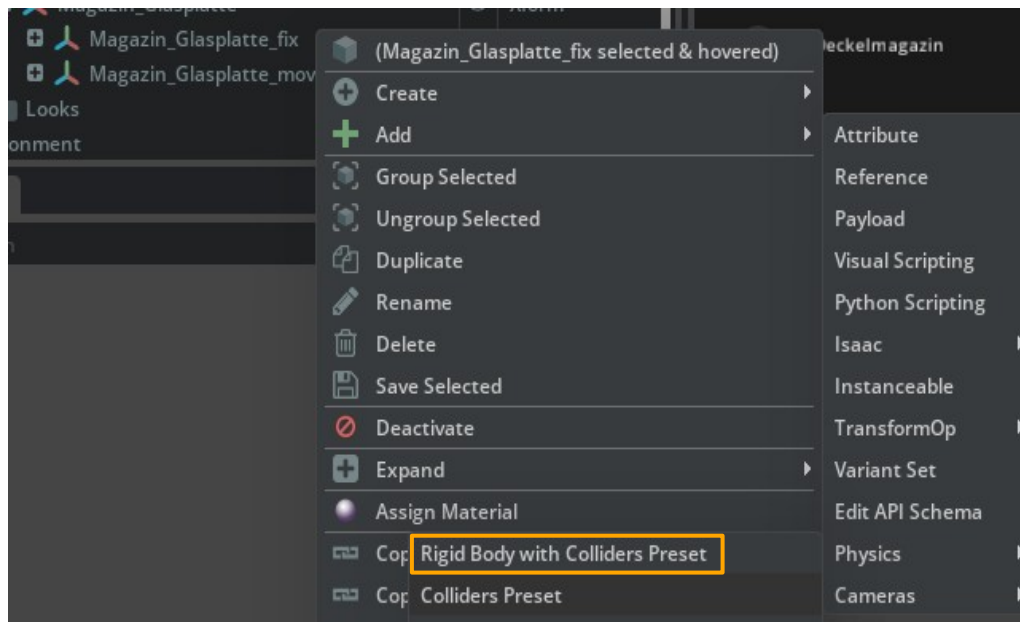
# Bewegliche Gelenkteile am Bsp. Magazin Glasplatte

1. Strg-Taste halten und alle beweglichen Teile im Modell auswählen
2. Rechtsklick auf eines der Teile und „Group Selected“ auswählen
3. Die erstellte Gruppe „Magazin\_Glasplatte\_move“ benennen



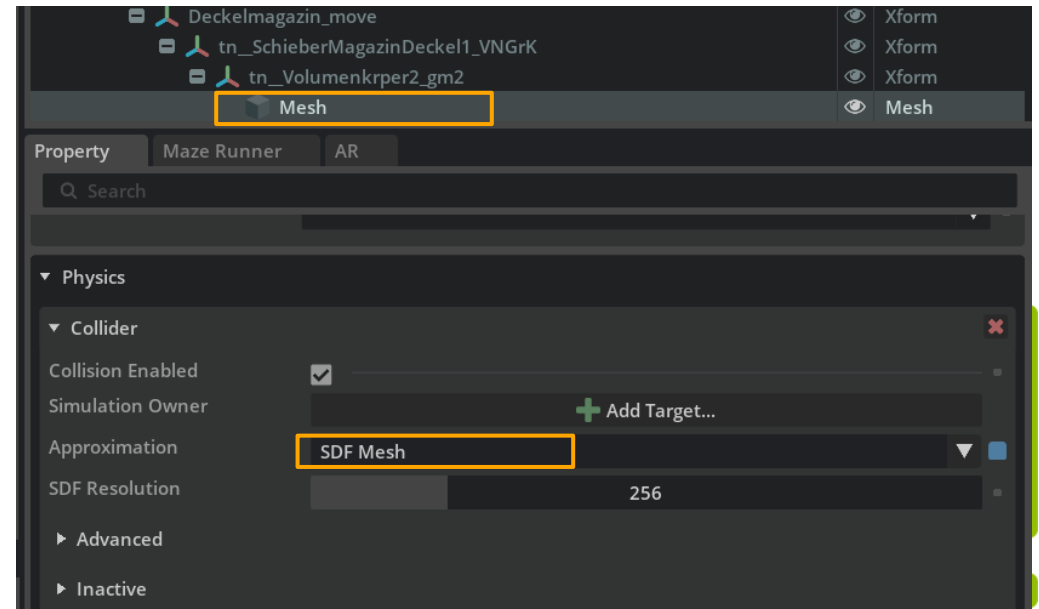
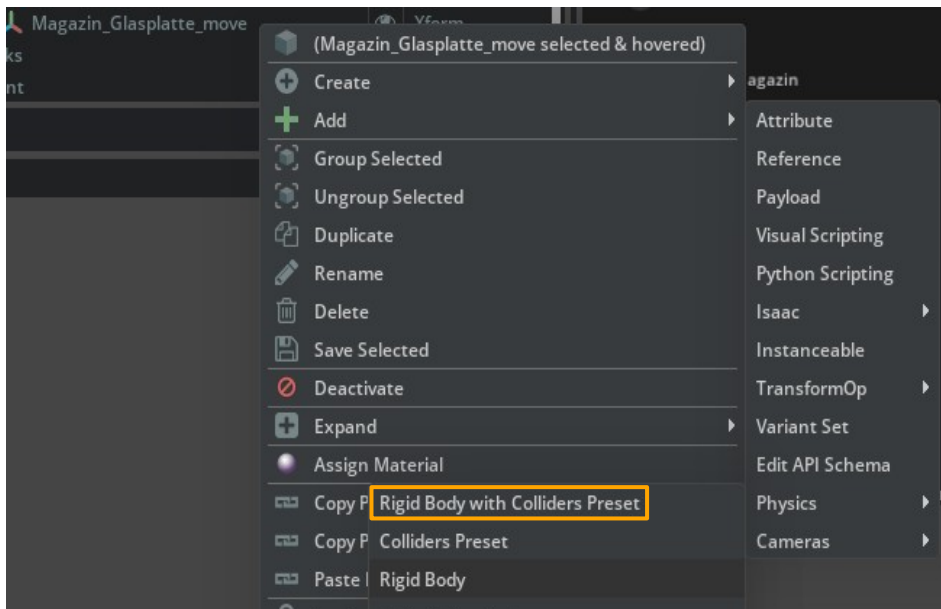
# Physikalische Eigenschaften starrer Gelenkteile

1. Rechtsklick auf bewegliche Gruppe „Add“ → „Physics“ → „Rigid Body“
2. In Property unter „Rigid Body“ „Kinematic Enabled“ aktivieren  
(So bleibt der Körper am Ort und fällt nicht durch die Grundplatte)



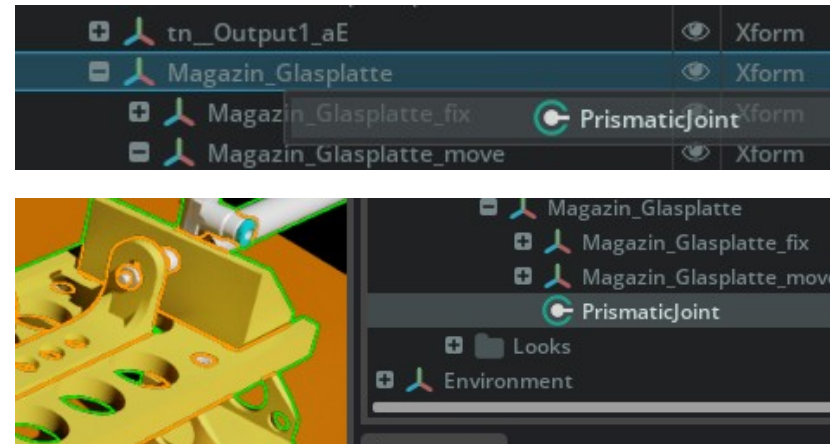
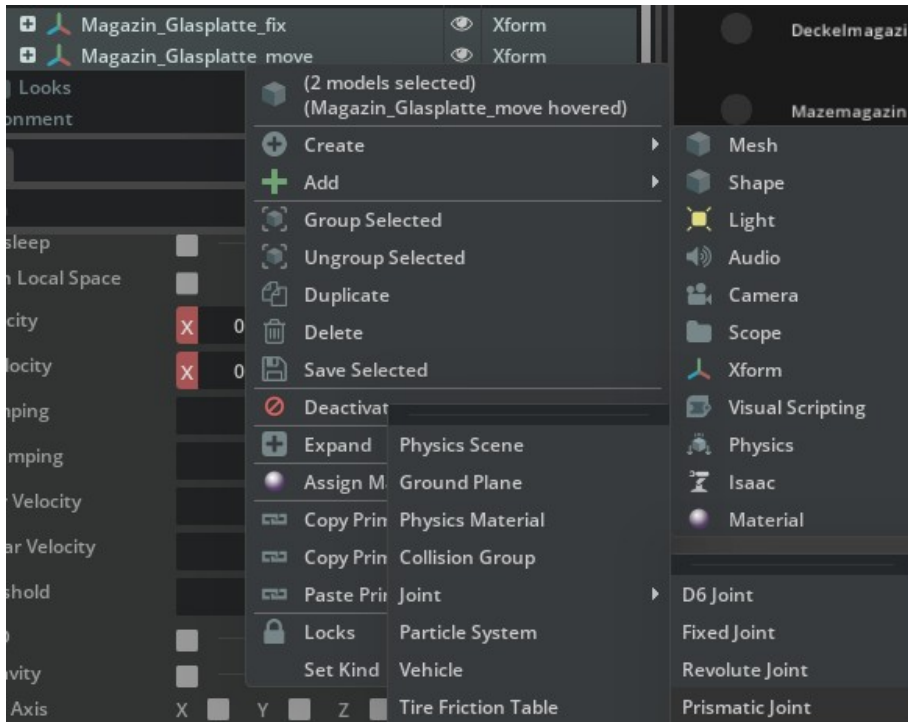
# Physikalische Eigenschaften beweglicher Körper

1. Rechtsklick auf bewegliche Gruppe „Add“ → „Physics“ → „Rigid Body with Collider presets“
2. Der „Collider“ beschreibt die Hülle die um das Mesh gelegt wird und bei Berührung mit anderen Körpern zur Berechnung verwendet wird und kann unter „Physics“ der jeweiligen „Mesh Datei“ eingesehen werden.
3. Für die höchste Genauigkeit empfiehlt sich „SDF Mesh“ oder „Convex Decomposition“ im Feld „Approximation“ auszuwählen.
4. Dies gilt ebenfalls für frei bewegliche Körper wie der Deckel.



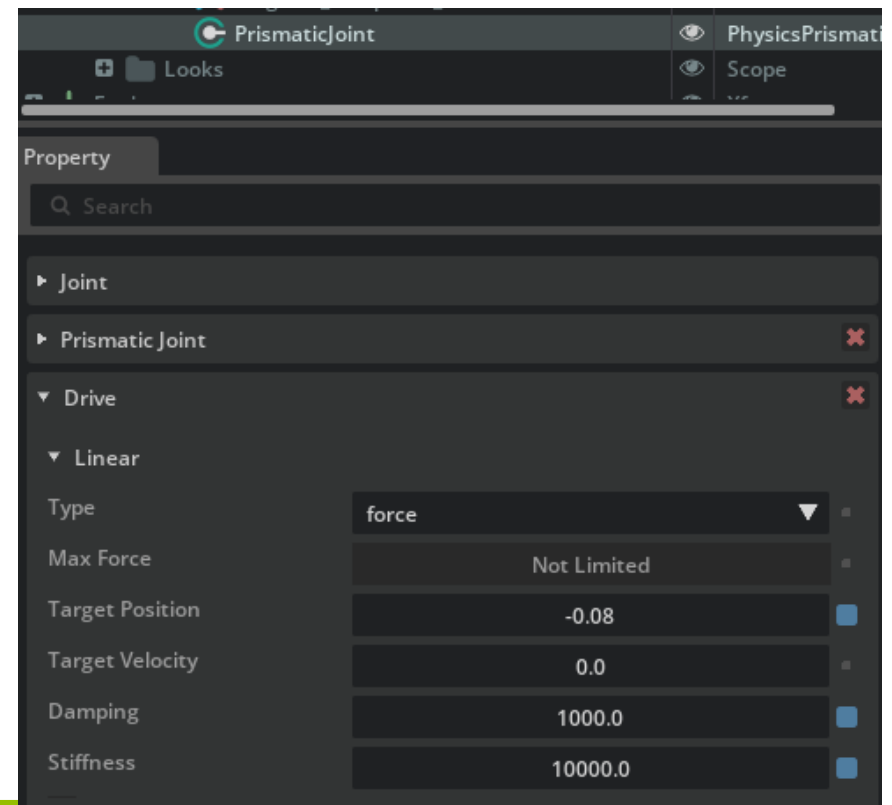
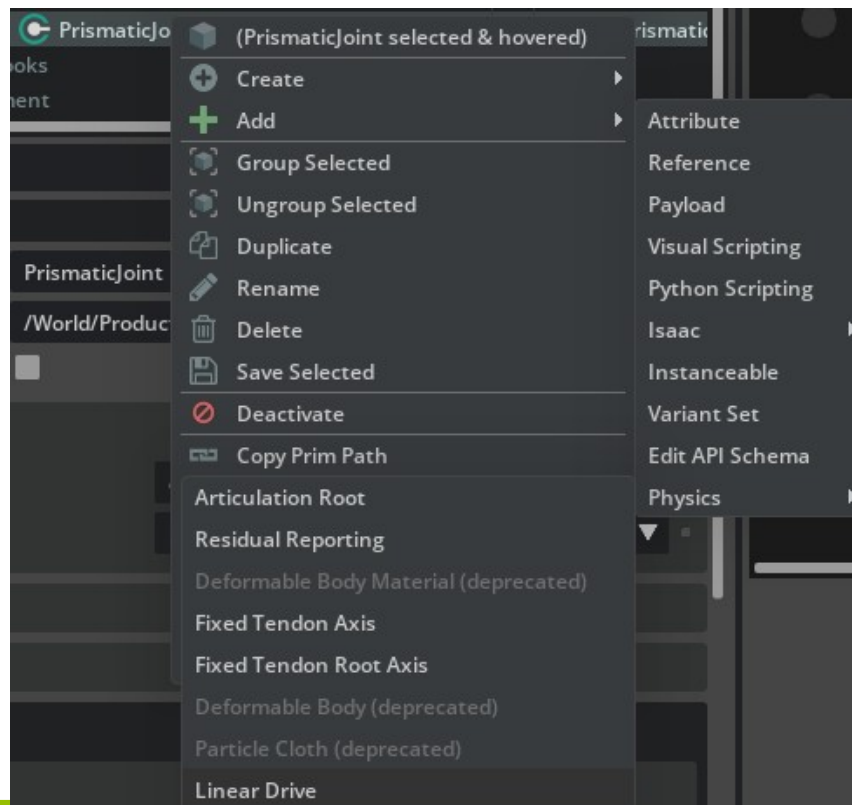
# Gelenk (Joint) hinzufügen

1. Beide Unterordner auswählen (\_move & \_fix) → Rechtsklick → Create → Physics → Joint → Prismatic Joint
2. Joint Eintrag per Drag and Drop in Hauptordner ablegen
3. Bei Auswahl des Joints sollte das Modell wie hier farblich markiert werden



# Antrieb einfügen

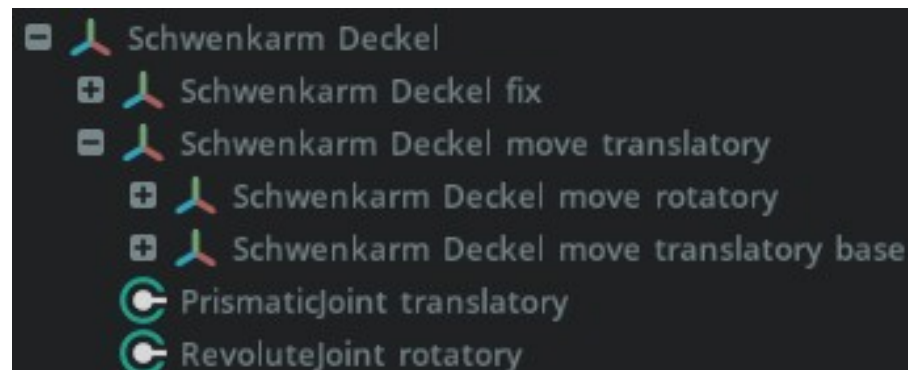
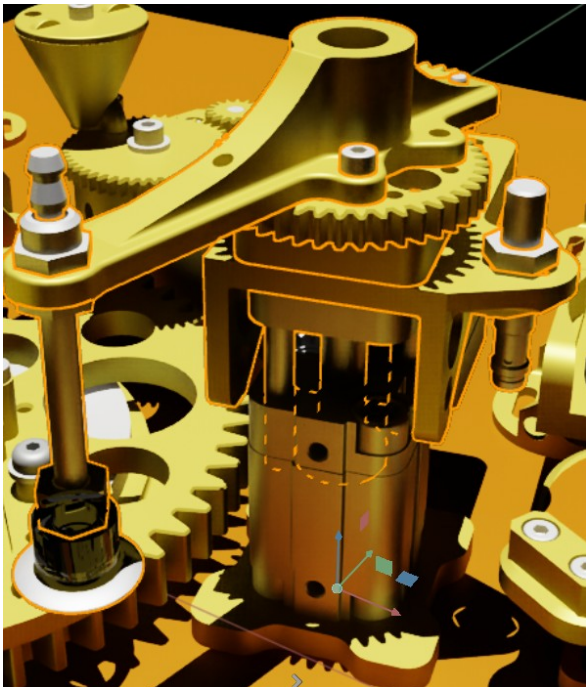
1. Rechtsklick auf Joint → „Add“ → „Physics“ → „Linear Drive“
2. In Property unter „Drive“ Werte wie im Bild übernehmen und ggf. anpassen
3. Funktion mit Play-Symbol oder Leertaste Testen





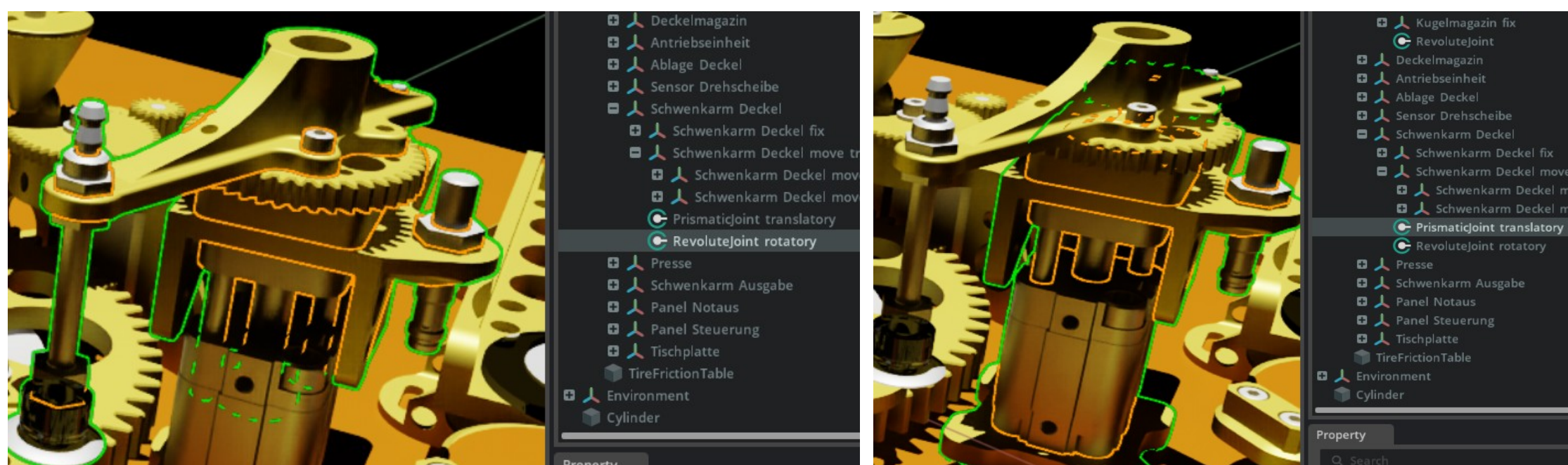
# Verschachtelte Gelenke

1. Für Verschachtelte Gelenke gibt es Details die beachtet werden müssen
2. Diese werden im Folgenden am Beispiel des Sauggreifer Gelenkarms aufgezeigt
3. Dieser verfügt über ein Drehgelenk im oberen Teil und ein Schubgelenk für den oberen Aufbau mit der entsprechenden Ordnerstruktur



# Verschachtelte Gelenke

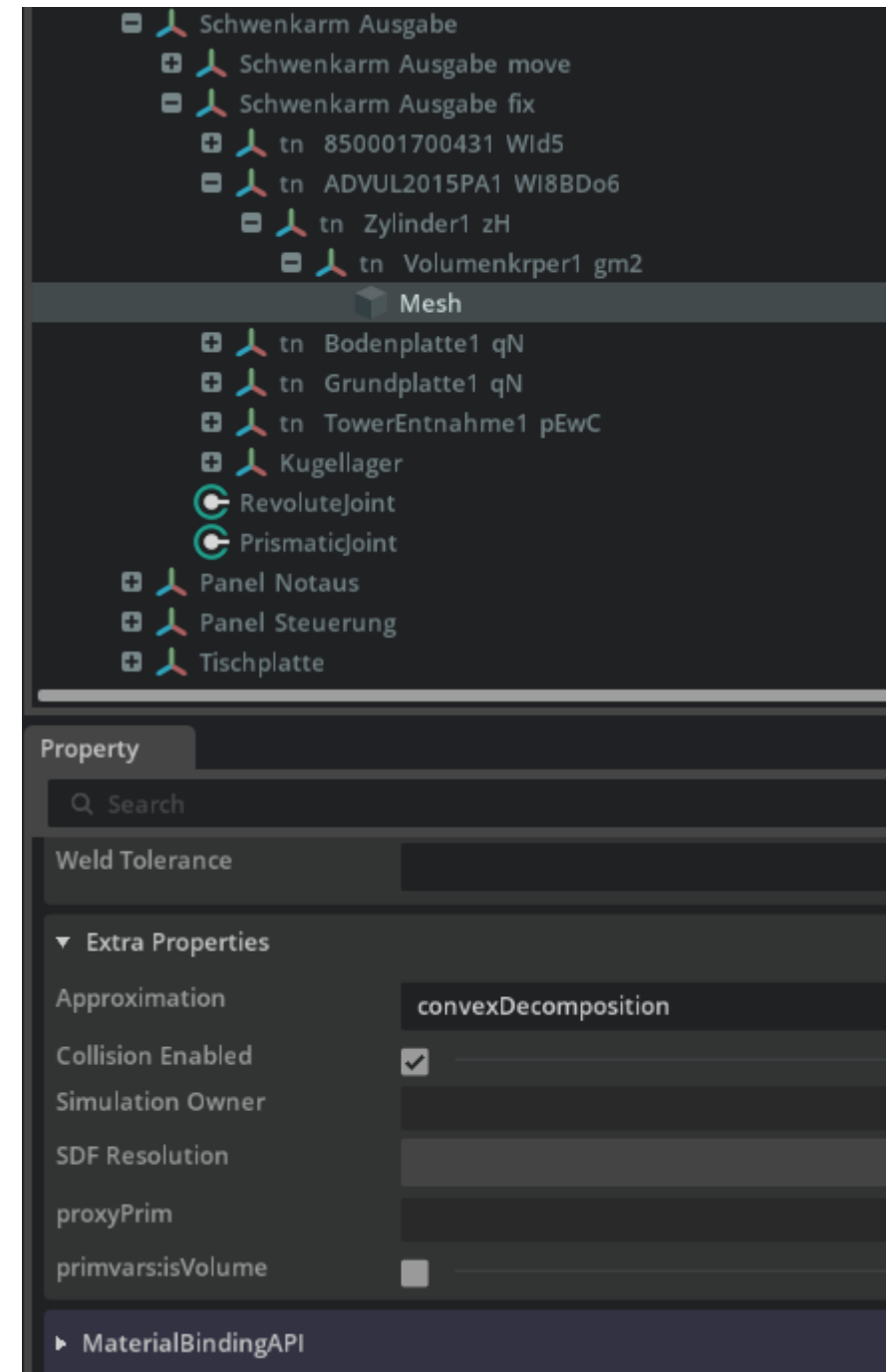
1. Für die Drehbewegung wird ein Gelenk zwischen beiden Baugruppen mit der Bezeichnung „move“ erstellt.
2. Für die Schubbewegung darf ein Gelenk nur zwischen der „Base“ und der nicht drehenden Trägerbaugruppe erstellt werden.





# Verschachtelte Gelenke

1. Das Mesh eines Körpers wird von der Physx Engine Approximiert.
2. „convexDecomposition“ löst das Mesh sehr genau auf und verhindert somit Kollisionen zwischen ungenauen Körpern welche über ihre Maße hinaus Approximiert wurden.



# Quelle

Labor Mechatronik (Trainingslevel Konstruktion und Inbetriebnahme einer Maze Runner Produktionslinie unter Einsatz interoperabler Edge- und Cloud-Technologien) SS2024

